异卷接口预留方案 - 简洁实用版

一、核心思路：事件钩子系统

最简单有效的方式是使用**事件钩子系统**，让异卷可以在关键点插入自己的逻辑。

基本架构

```
# game_core.py - 游戏核心类
class GameCore:
    def __init__(self):
        self.hooks = {}  # 存储所有钩子
        self.register_default_hooks()

    def register_hook(self, event_name, callback, pri
        """注册事件钩子"""
        if event_name not in self.hooks:
            self.hooks[event_name] = []
        self.hooks[event_name].append((callback, pric
        # 按优先级排序
        self.hooks[event_name].sort(key=lambda x: x[1

    def trigger_hook(self, event_name, *args, **kwarg
        """触发事件钩子"""
        if event_name not in self.hooks:
            return None

        result = None
```

清言·AI生成

```
        for callback, _ in self.hooks[event_name]:
            result = callback(*args, **kwargs)
            # 如果返回False，阻止后续执行
            if result is False:
                break
        return result

    def register_default_hooks(self):
        """注册默认事件"""
        # 游戏流程关键点
        self.hooks = {
            'on_game_start': [],
            'on_round_start': [],
            'on_turn_start': [],
            'on_card_played': [],
            'on_player_action': [],
            'on_round_end': [],
            'on_game_end': []
        }
```

二、逻辑接口预留

2.1 游戏流程关键点插入钩子

在游戏核心逻辑的关键位置插入钩子调用点：

示例：回合流程

```
class GameCore:
    def start_round(self):
```

清言·AI生成

```
        """开始回合"""
        # 触发回合开始钩子
        self.trigger_hook('on_round_start', self.curr

        # 默认逻辑
        self.deal_cards()

        for player in self.players:
            # 触发玩家回合开始钩子
            self.trigger_hook('on_turn_start', player

            # 玩家行动
            action = player.take_action()

            # 触发玩家行动钩子
            should_continue = self.trigger_hook('on_p

            # 如果钩子没有阻止，执行默认逻辑
            if should_continue is not False:
                self.execute_action(player, action)

        # 触发回合结束钩子
        self.trigger_hook('on_round_end', self.curren
```

2.2 提供数据访问接口

异卷需要访问和修改游戏数据，提供简单的接口：

数据访问接口

清言·AI生成

```
class GameCore:
    def get_game_state(self):
        """获取游戏状态（只读）"""
        return {
            'round': self.current_round,
            'players': [p.to_dict() for p in self.pla
            'current_player': self.current_player.id,
            'deck_size': len(self.deck)
        }

    def modify_game_state(self, key, value):
        """修改游戏状态"""
        if hasattr(self, key):
            setattr(self, key, value)
            return True
        return False

    def add_player_attribute(self, player_id, attr_na
        """给玩家添加自定义属性"""
        player = self.get_player(player_id)
        if player:
            setattr(player, attr_name, value)
            return True
        return False
```

三、UI接口预留

3.1 UI组件系统

为UI提供可扩展的组件接口：

清言·AI生成

UI管理器

```
# ui_manager.py
class UIManager:
    def __init__(self):
        self.custom_components = {}
        self.ui_hooks = {}

    def register_component(self, component_id, compor
        """注册自定义UI组件"""
        self.custom_components[component_id] = compor

    def register_ui_hook(self, hook_name, callback):
        """注册UI钩子"""
        if hook_name not in self.ui_hooks:
            self.ui_hooks[hook_name] = []
        self.ui_hooks[hook_name].append(callback)

    def render_custom_ui(self, ui_type, *args, **kwar
        """渲染自定义UI"""
        # 先触发钩子
        for callback in self.ui_hooks.get(ui_type, []
            custom_ui = callback(*args, **kwargs)
            if custom_ui:
                return custom_ui

        # 返回默认UI
        return self.render_default_ui(ui_type, *args,

    def get_player_ui_elements(self, player_id):
        """获取玩家的UI元素"""
        elements = []
```

```
        # 触发钩子让异卷添加UI元素
        for callback in self.ui_hooks.get('player_ui
            custom_elements = callback(player_id)
            elements.extend(custom_elements or [])

        return elements
```

3.2 前端集成示例

JavaScript端集成

```
// game-ui.js
class GameUI {
    constructor() {
        this.customComponents = new Map();
        this.uiModifiers = new Map();
    }

    // 注册自定义UI组件
    registerComponent(name, component) {
        this.customComponents.set(name, component);
    }

    // 注册UI修改器
    registerUIModifier(elementId, modifier) {
        if (!this.uiModifiers.has(elementId)) {
            this.uiModifiers.set(elementId, []);
        }
        this.uiModifiers.get(elementId).push(modifie
    }
```

```
    // 渲染时应用修改
    renderElement(elementId, defaultRender) {
        let element = defaultRender();

        // 应用所有修改器
        const modifiers = this.uiModifiers.get(elemer
        for (const modifier of modifiers) {
            element = modifier(element) || element;
        }

        return element;
    }

    // 示例：异卷添加玩家状态显示
    addPlayerStatusUI(playerId, statusHTML) {
        const playerElement = document.getElementByIc
        if (playerElement) {
            const statusDiv = document.createElement(
            statusDiv.className = 'custom-status';
            statusDiv.innerHTML = statusHTML;
            playerElement.appendChild(statusDiv);
        }
    }
}
```

四、异卷开发示例

4.1 异卷基本结构

异卷模板

清言·AI生成

```
# my_module.py
{
    "name": "我的异卷",
    "version": "1.0.0",
    "author": "开发者",
    "dependencies": [],
    "description": "示例异卷"
}

# 异卷主类
class MyModule:
    def __init__(self, game_core, ui_manager):
        self.game = game_core
        self.ui = ui_manager
        self.register_hooks()

    def register_hooks(self):
        """注册所有钩子"""
        # 注册逻辑钩子
        self.game.register_hook('on_round_start', sel
        self.game.register_hook('on_player_action', s

        # 注册UI钩子
        self.ui.register_ui_hook('player_ui', self.ac

    def on_round_start(self, round_num):
        """回合开始时的逻辑"""
        # 修改游戏规则
        if round_num == 1:
            # 第一回合给所有玩家添加额外属性
            for player in self.game.players:
                self.game.add_player_attribute(player

    def on_player_action(self, player, action):
```

```
        """玩家行动时的逻辑"""
        # 拦截特定行动
        if action.type == 'special_move':
            # 执行自定义逻辑
            self.handle_special_move(player, action)
            return False  # 阻止默认执行
        return True  # 继续默认执行

    def add_custom_ui(self, player_id):
        """添加自定义UI元素"""
        player = self.game.get_player(player_id)
        if hasattr(player, 'bonus_points'):
            return [{
                'type': 'div',
                'class': 'bonus-display',
                'content': f'奖励点数: {player.bonus_p
            }]
        return []

# 异卷必须提供的初始化函数
def initialize(game_core, ui_manager):
    return MyModule(game_core, ui_manager)
```

4.2 简单示例：添加新卡牌类型

异卷实现

```
# special_cards_module.py
{
    "name": "特殊卡牌",
    "version": "1.0.0"
```

清言·AI生成

```
}

class SpecialCardsModule:
    def __init__(self, game_core, ui_manager):
        self.game = game_core
        self.ui = ui_manager
        self.register_hooks()

    def register_hooks(self):
        # 在卡牌使用前检查
        self.game.register_hook('on_player_action', s

        # 添加卡牌信息UI
        self.ui.register_ui_hook('card_info', self.sh

    def check_special_card(self, player, action):
        """检查是否使用特殊卡牌"""
        if action.type == 'play_card':
            card = action.card
            if card.type == 'special':
                # 执行特殊卡牌效果
                self.execute_special_effect(player, c
                return False  # 不执行默认出牌逻辑
        return True

    def execute_special_effect(self, player, card):
        """执行特殊效果"""
        if card.name == '双倍卡':
            # 下回合双倍得分
            self.game.add_player_attribute(
                player.id,
                'double_points_next_round',
                True
            )
```

```
    def show_card_info(self, card):
        """显示卡牌额外信息"""
        if card.type == 'special':
            return f"
特殊卡牌效果: {card.effect}
"
        return None

def initialize(game_core, ui_manager):
    return SpecialCardsModule(game_core, ui_manager)
```

五、实用建议

5.1 接口设计原则

- **简单优先**：只暴露必要的接口，不要过度设计
- **事件驱动**：用事件钩子实现扩展点，而不是复杂的继承体系
- **数据隔离**：异卷通过接口访问数据，不直接操作内部状态
- **错误处理**：钩子执行要有异常捕获，避免异卷崩溃影响主程序

5.2 关键接口列表

逻辑接口：

- `register_hook(event, callback)` - 注册事件钩子
- `get_game_state()` - 获取游戏状态
- `modify_game_state(key, value)` - 修改游戏状态
- `add_player_attribute(player_id, name, value)` - 添加玩家属性

清言·AI生成

UI接口：

- `register_component(id, component)` - 注册UI组件
- `register_ui_hook(hook_name, callback)` - 注册UI钩子
- `add_player_status_ui(player_id, html)` - 添加玩家状态UI

5.3 开发流程

1. **先开发核心功能**：不要一开始就考虑扩展
2. **识别扩展点**：在关键位置插入钩子调用
3. **提供必要接口**：根据实际需要暴露数据和方法
4. **逐步完善**：根据异卷开发需求逐步增加接口

> **实现清单**
>
> ```
> # 核心系统需要实现的最小接口集
>
> class GameCore:
>  # 1. 钩子系统
>  register_hook(event_name, callback, priority)
>  trigger_hook(event_name, *args, **kwargs)
>
>  # 2. 数据访问
>  get_game_state()
>  modify_game_state(key, value)
>  get_player(player_id)
>  add_player_attribute(player_id, attr_name, value)
>
>  # 3. 游戏流程关键点的钩子调用
>  # on_game_start, on_round_start, on_turn_start
>  # on_player_action, on_round_end, on_game_end
> ```

```
class UIManager:
    # 1. UI组件注册
    register_component(component_id, component_class)

    # 2. UI钩子
    register_ui_hook(hook_name, callback)

    # 3. 渲染接口
    render_custom_ui(ui_type, *args, **kwargs)
    add_player_status_ui(player_id, html_content)
```

六、总结

通过**事件钩子系统**，您可以在不增加复杂度的情况下，为异卷提供充分的扩展能力。关键点：

1. 在游戏流程关键点调用钩子
2. 提供必要的数据访问接口
3. UI系统支持自定义组件和修改器
4. 保持接口简单，按需扩展

这种方式既能满足功能需求，又不会过度设计，适合快速开发和迭代。

清言·AI生成